



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

Μάθημα: Ευφυής Έλεγχος

Απαλλακτική εργασία

Συμμετέχοντες:

Γιακουμάτος Γεώργιος - ΑΜ: 1092874

Σάλιαρης Γεώργιος - ΑΜ: 1092584

**Υπεύθυνος καθηγητής: Κωνσταντίνος
Χατζηλυγερούδης**

Ακαδημαϊκό έτος: 2024-25

1-Σύντομη παρουσίαση της εργασίας

Σκοπός της παρούσης εργασίας είναι η εφαρμογή του **Quality-Diversity** αλγορίθμου **SAIL** (**Surrogate-Assisted Illumination**) για ένα σχετικά απλό τετράποδο ρομπότ το οποίο και προσομοιώνουμε. Ειδικότερα, μέσω του εν λόγω αλγορίθμου το ρομπότ περπατάει με διάφορους τρόπους, μέσω της παρακάτω λογικής η οποία συνοψίζει και την ουσία των *Quality-Diversity* αλγορίθμων:

- 1) Χώρισε το πλήθος των δυνατών συμπεριφορών του ρομπότ σε κουτάκια
- 2) Σε κάθε κουτάκι δοκίμασε κάτι διαφορετικό, εάν είναι καλό τότε, τοποθέτησέ το στο κουτάκι αυτό.
- 3) Επίλεξε κινήσεις οι οποίες φαίνονται και καλές αλλά και δεν έχουν εξερευνηθεί μέχρι στιγμής.

2-Παρουσίαση βασικών εννοιών οι οποίες αφορούν στον SAIL αλγόριθμο

Παρακάτω παραθέτουμε μια γρήγορη παρουσίαση των πιο σχετικών εννοιών αναφορικά με τον SAIL αλγόριθμο *per se* αλλά και των *Quality-Diversity* αλγορίθμων εν γένει.

α)Βασικές έννοιες των QD/MAP ELITE αλγορίθμων

1. Quality-Diversity (QD): Αλγόριθμοι που, αντί να αναζητούν μία μόνο «καλύτερη» λύση, προσπαθούν να δημιουργήσουν **μεγάλη συλλογή ποιοτικών και ποικίλων λύσεων**. Ιδανικοί για προβλήματα όπου υπάρχουν πολλαπλές λύσεις και όχι μια και μόνο.

2. Behavioral Descriptor (BD): Πρόκειται για ένα σετ από αριθμούς οι οποίοι χαρακτηρίζουν μια λύση, χωρίς να προσδιορίζουν απαραίτητα πόσο «καλή» είναι η συγκεκριμένη λύση. Ορίζουν το **behaviour space** (χώρο συμπεριφοράς) όπου τοποθετούμε τις λύσεις μας.

3. Simulation: Προσομοίωση του πραγματικού συστήματος, όπως αυτή υποδεικνύεται από μια λύση (**genotype**).

4. Cell/Niche: Είναι ένα τετράγωνο στο **BD grid** συμπεριφορών. Ανάλογα με τη διάσταση του **BD** μπορεί να εκτείνεται σε περισσότερες των 2 διαστάσεις. Στη συγκεκριμένη περίπτωση, κάθε κελί αντιστοιχίζεται και σε ένα διαφορετικό τρόπο του ρομπότ να «περπατάει».

5. Fitness function: Μας λέει **πόσο καλή είναι μια λύση**.

6. Archive/Map: Ολόκληρο το **grid** το οποίο αποθηκεύει την καλύτερη λύση σε κάθε *cell*.

6. Elite: Η καλύτερη λύση που βρήκαμε για ένα δεδομένο **cell**.

7. Coverage: Πόσα *cells* περιέχουν μια **elite** λύση (εκπεφρασμένη σε ποσοστό % επί των συνολικών *cells*).

8. Variation/Mutation: Όπως και στους εξελικτικούς/γενετικούς αλγορίθμους, συνιστά μια μικρή αλλαγή την οποία εφαρμόζουμε σε μια *elite* λύση προκειμένου να δημιουργήσουμε μια νέα υποψήφια (***candidate***) λύση, προκειμένου να ενισχυθεί το γενικότερο ***diversity*** των λύσεων που έχουμε στο ***archive***.

9. Descriptor Space: Το σύστημα συντεταγμένων του οποίου οι άξονες είναι οι ***behavioural descriptors*** που επιλέξαμε. Η διαφορά του εν συγκρίσει με το *archive*, είναι ότι το τελευταίο είναι μια διακριτοποίηση του πρώτου.

10. Map-Elites (Multi-dimensional Archive of Phenotypic Elites): Χωρίζουν το *descriptor space* σε ένα *grid* συμπεριφορών και κρατάνε την καλύτερη λύση ανά *cell*, εφαρμόζοντας σε αυτές παράλληλα και ***mutation*** προκειμένου να έχουμε περισσότερο *exploration*.

11. Seed/Initialization: Πρόκειται για το **πρώτο βήμα** όταν τρέχει ένας QD αλγόριθμος όπου δημιουργούμε ένα αρχικό σετ από διάφορες λύσεις προκειμένου να τις εντάξουμε στο *archive*.

12. QD-Score: Αριθμός ο οποίος συνδυάζει ***coverage* και *quality*** (άθροισμα όλων των ***fitness*** τιμών για όλες τις *elite* λύσεις).

b) Έννοιες που εισάγει ο SAIL αλγόριθμος:

13. Surrogate Model: Πρόκειται για μια σχετικά γρήγορη και φθηνή εκτίμηση της *fitness function*, η οποία δεδομένης της πολυπλοκότητάς της και του πλήθους των λύσεων που καλείται να αξιολογήσει, ενδεχομένως να μην είναι εφαρμόσιμη σε πραγματικό χρόνο.

14. Gaussian process: Ένα μοντέλο το οποίο έχει ως έξοδο δύο πράγματα: ένα αναμενόμενο **score** (μ) και μια **αβεβαιότητα** (σ). Ουσιαστικά μαθαίνουν από λίγες και ακριβές προσομοιώσεις και μετά προβλέπουν το πόσο καλή είναι μια δεδομένη *candidate* λύση όπως επίσης και την αβεβαιότητά της. Πρόκειται για έναν ειδικό τύπο **surrogate model** που υπό μια πρώτη ματιά δεν έχει παραμέτρους προς ρύθμιση.

15. Upper-Confidence Bound (UCB): $UCB(x) = \mu(x) + \beta\sigma(x)$ (2.1), όπου $\mu(x)$ και $\sigma(x)$ είναι οι εξοδοί του *surrogate model* για μια υποψήφια λύση x (β είναι μια παράμετρος η οποία ελέγχει το *balance* ανάμεσα στο **exploration** και στο **exploitation**). Ο δε SAIL αλγόριθμος, επιλέγει για κάθε κελί τη *candidate solution* με το υψηλότερο **UCB** προκειμένου να επιλέξει ποιες λύσεις θα προσομοιωθούν.

16. Prediction Map: Ένα δευτερεύον **archive** το οποίο δημιουργείται αφ'ότου ολοκληρωθεί/σταματήσει ο αλγόριθμος και περιέχει για κάθε κελί το μοναδική λύση (**genotype**) την οποία το *surrogate model* προβλέπει ότι θα έχει σχετικά μεγάλη απόδοση.

17. Acquisition Map: Για κάθε επανάληψη του αλγορίθμου, το *surrogate model* αξιολογεί ένα αρκετά μεγάλο *batch* από **random** λύσεις και κρατάει εκείνη με το υψηλότερο **UCB**.

18. Precise Evaluation: Είναι οποιοσδήποτε πραγματικός υπολογισμός της *fitness function*.

3-Σύντομη παρουσίαση του αλγορίθμου και του σχετικού κώδικα

Στον σχετικό κώδικα (βλ. αρχείο *QD_dummy_SAILv7.py*) όπου και εφαρμόζουμε τον αλγόριθμο ακολουθούμε τα εξής βήματα:

Βήμα 1-Ορίζουμε την υποψήφια λύση-genotype

Ουσιαστικά, μια υποψήφια λύση αποτελείται από ένα διάνυσμα τεσσάρων πλατών $A_i, i = 1,2,3,4$. Κάθε A_i αποτελεί το μέγιστο «άνοιγμα» μιας άρθρωσης του τετράποδου ρομπότ μας που ελέγχεται από τον αντίστοιχο σερβοκινητήρα. Κάθε δε σερβοκινητήρας ακολουθεί τον παρακάτω νόμο ελέγχου (*control command*):

$$\theta_i(t) = A_i \cdot \sin(2\pi t / T) \quad (T = 1,5 s) \quad (3.1).$$

Ο δε **PD** ελεγκτής κάθε σερβοκινητήρα μετατρέπει το σφάλμα θέσης σε ροπή και έτσι η επιθυμητή ταχύτητα (άρα και το πώς θέλουμε να κινείται το ρομπότ) προκύπτουν αυτόματα.

Βήμα 2-Διαμόρφωση descriptor space

Επιλέγουμε ως *behavioural descriptors* **ταχύτητα** και **effort** («ενέργεια») και χωρίζουμε το *descriptor space* με τέτοιο τρόπο έτσι ώστε να έχεις διαστάσεις 25x25. Ο κάθε δείκτης κελίου έχει ως εξής:

$$i = \text{floor} \left((\text{speed} - 0) * \frac{24}{2} \right) \quad (3.2)$$

$$j = \text{floor} \left((\text{effort} - 0) * \frac{24}{2} \right) \quad (3.3)$$

Έτσι, έχουμε $25 \times 25 = 625$ (3.4) **θήκες (bins)** όπου μπορούμε να αποθηκεύσουμε διαφορετικές συμπεριφορές.

Βήμα 3-Δημιουργία αρχικού πληθυσμού 60 τυχαίων genotypes

Για κάθε δείγμα εκτελούμε μια πλήρη προσομοίωση και υπολογίζουμε τις παρακάτω ποσότητες:

$$speed = \frac{\text{μετατόπιση}}{1.5} \quad (3.5)$$

$$effort = 2 \cdot \text{mean}(A^2) \quad (3.6)$$

$$fitness = speed - 0.1 \cdot effort \quad (3.7)$$

Βήμα 4-Χρησιμοποιούμε Mattern 5/2 + WhiteNoise:

$$\mu(x) = k(x, X) \cdot (K + \sigma_n^2 I)^{-1} \cdot y \quad (3.8)$$

$$\sigma^2(x) = k(x, x) - k(x, X) \cdot (K + \sigma_n^2 I)^{-1} \cdot k(X, x) \quad (3.9)$$

Οι παραπάνω αποτελούν τις εξισώσεις που αντιστοιχούν στο GP.

Για την παραπάνω εξίσωση:

Symbol	Role
x	Test (query) input where you want a prediction. May be a scalar or a length- d feature vector.
X	Training inputs collected in one matrix: $X = [x^{(1)}, x^{(2)}, \dots, x^{(N)}]^\top$.
y	Training targets/outputs corresponding to each row of X .
K	Training covariance matrix with entries $K_{ij} = k(X_i, X_j)$.
$k(x, X)$	Row vector of covariances between the test point x and every training point.
$k(X, x)$	Column-vector transpose of $k(x, X)$.
$k(x, x)$	Prior variance of the latent function at x .
σ_n	Observation-noise std. dev. (variance = σ_n^2).
I	Identity matrix ; same size as K ($N \times N$). Adding $\sigma_n^2 I$ places independent noise on each training point only along the diagonal.



Βήμα 5- Δημιουργία 400 υποψήφιων λύσεων

Λαμβάνουμε 400 ομοιόμορφα δείγματα στο διάστημα $[-1,1]$ τα οποία σχετικά γρήγορα αξιολογεί το *Gaussian Process*.

Βήμα 6-Υπολογισμός UCB

$$UCB(x) = mu(x) + \beta \cdot sigma(x) \quad (\beta = 2.0) \quad (3.10).$$

όπου να σημειώσουμε ότι στην παραπάνω σχέση μικρή τιμή της παραμέτρου β , σημαίνει ότι κάνουμε ***favour to exploitation σε σχέση με το exploration*** και μια μεγάλη τιμή του β , σημαίνει ακριβώς το ανάποδο.

Βήμα 7-Φιλτράρισμα λύσεων

Για κάθε κελί του *descriptor space*, κρατάμε μόνο τον υποψήφιο με το μεγαλύτερο *UCB* (σε περίπτωση που δύο τυχαίες λύσεις, αντιστοιχούν στο ίδιο κελί). Με αυτόν τον τρόπο **εξασφαλίζουμε διαφορετικές συμπεριφορές** πριν τις ακριβείς προσομοιώσεις για κάθε υποψήφια λύση.

Βήμα 8-Επιλογή 12 κορυφαίων λύσεων για «πραγματική» προσομοίωση

Ουσιαστικά προσομοιώνουμε τις 12 κορυφαίες λύσεις (εκείνες που έχουν το υψηλότερο *UCB*), υπολογίζουμε *fitness* και κάνουμε *update* το *grid* και το *dataset* του *GP*.

Βήμα 9-«Πραγματική» αξιολόγηση λύσεων και ενημέρωση του *grid*

Επί της ουσίας:

- Μετράμε νέο *speed*, *effort*, *fitness*.

- Αν βάσει του *fitness score* βελτιώνεται το *elite* του κάθε κελιού ενημερώνουμε το πλέγμα.

- Προσθέτουμε την πλειάδα (***genotype, fitness***) στο ***training set*** και έτσι το μοντέλο *GP* μαθαίνει καλύτερα στον επόμενο κύκλο (ή τουλάχιστον έτσι πρέπει να γίνεται).

Βήμα 10-Καταγραφή μετρικών

$$coverage = \frac{\text{γεμάτα_κελιά}}{625} \quad (3.11) - \text{αντιστοιχεί σε έναν δείκτη exploration}$$

$$mean_{fitness} = \text{μέσος όρος fitness γεμάτων κελιών} \quad (3.12)$$

$$- \text{αντιστοιχεί σε έναν δείκτη exploitation} \quad (3.13)$$

Βήμα 11-Επαναλαμβάνουμε τα βήματα 4-10 για 60 κύκλους

Εν τέλει (αφού τρέξουμε τον κώδικα) έχουμε:

1. Ένα αρχείο που περιέχει πολλαπλές elite λύσεις βηματισμού του ρομπότ
2. Εκπαιδευμένο GP
3. Διαγράμματα *heat-map*, *scatter speed/effort*, *learning curve* και ένα *3D plot*

4-Περαιτέρω εξήγηση του σχετικού κώδικα

1. PID μέσα σε κάθε σερβοκινητήρα

Έχουμε το εξής *setup*: κινητήρας + κωδικοποιητής + πλακέτα *PD*. Ουσιαστικά, στέλνουμε μια γωνία στόχου θ_{target} στον **encoder**. Ο *PD* με την σειρά του το συγκρίνει με τη μετρούμενη γωνία και οδηγεί το ρεύμα μέχρι το σφάλμα ≈ 0 . Επομένως, ο κινητήρας «υπολογίζει» μόνος του την **απαιτούμενη ταχύτητα και ροπή**.

2. Τι θα συμβεί αν προσθέσουμε περισσότερους behavioural descriptors;

Το πλέγμα γίνεται 3-D /4-D κλπ -> πολλά περισσότερα κελιά προς συμπλήρωση -> **πολλές ακόμη αξιολογήσεις ή ένας πιο αραιός χάρτης**.

Πλεονέκτημα: πιο πλούσια γνώση (διαφορετικές και ποιο ποιοτικές συμπεριφορές)

Μειονέκτημα: **Κατάρτα του dimensionality**—η κάλυψη (*coverage*) πέφτει αρκετά και η πολυπλοκότητα του προβλήματος αυξάνεται εκθετικά.

3. Τι αποθηκεύεται στο .csv αρχείο;

Για κάθε **filled cell**: ταχύτητα, *effort*, *fitness* και τα τέσσερα πλάτη (*amp_0...amp_3*) βάσει των οποίων προκύπτουν και οι πολλαπλές συμπεριφορές (τρόποι βαδίσματος) του ρομπότ.

4. Για ποιο λόγο επιλέγουμε grid διαστάσεων 25 x 25

Δεν υπάρχει κάποιος συγκεκριμένος λόγος-«εμπειρικά» επιλέγουμε τις διαστάσεις αυτές.

5. Δείκτης κάθε κελιού

Μας δείχνει σε ποιο ακριβώς κελί του *descriptor space* εμπίπτει μια συμπεριφορά του ρομπότ.

6. Πλήρης προσομοίωση στο Βήμα 3

Τρέχουμε το «**ακριβό**» (υπό την άποψη των υπολογιστικών *resources* που απαιτούνται) ***simulation*** το εκάστοτε γονιδίωμα, μετράμε την πραγματική ταχύτητα και *effort*.

7. Fitness function

$$fitness = speed - 0.1 * effort \quad (4.1)$$

8. Τι δείχνει κάθε plot

- a) *Heat map*: πού βρίσκονται (στο *grid*) καλοί τρόποι βαδίσματος του ρομπότ.
 - b) *Scatter*: (*speed*, *effort*) σημεία μαζί επίσης με το αντίστοιχο *fitness* (το οποίο αναπαρίσταται από το χρώμα)
 - c) *Learning curves*: *iterations* (x άξονας) vs *coverage* και *average fitness* (y-άξονας)
 - d) *3D plot*: ένα **3D «cloud»** των μεγεθών (*speed*, *effort*, *fitness*), το οποίο μας δίνει ένα insight για το ***fitness landscape***.
-

9. Τι είναι το effort

Ουσιαστικά είναι ένα μέγεθος το οποίο μας δείχνει πόση ενέργεια καταναλώθηκε από κάθε κινητήρα του ρομπότ ανά απόσταση που διένυσε το τελευταίο.

10. Budget

Budget = ο σχετικά περιορισμένος αριθμός πραγματικών προσομοιώσεων που μπορούμε να τρέξουμε λόγω των διαθέσιμων υπολογιστικών *resources*.

11. Kernel/*Matérn*

Kernel == *covariance function*: **μας δείχνει πόσο παρόμοιοι είναι δύο *genotypes***. Στον κώδικα κάνουμε χρήση ενός ***Matérn 5/2 kernel*** που έχει ως εξής:

$$k(r = |x - x'|) = \sigma^2 * \left(\frac{1 + \sqrt{5}r}{l} + 5 \frac{r^2}{3l^2} \right) * \exp\left(\frac{-\sqrt{5}r}{l}\right) (4.2).$$

Όπου l είναι μια παράμετρος η οποία ουσιαστικά μας λέει πόσο μπορούν να διαφέρουν δύο *genotypes* x και x' πριν το GP τις θεωρήσει ασυσχέτιστες (ότι διαφέρουν πάρα πολύ δηλαδή).

12. GP dataset

Το ***training*** περιλαμβάνει όλες τις λύσεις που προσομοιώνονται όπως επίσης και τις αντίστοιχες τιμές *fitness*. Κάνουμε train το GP πάνω σε αυτό το *dataset*.

13. Ποιο ρομπότ χρησιμοποιείται/προσομοιώνεται

Χρησιμοποιείται ένα έτοιμο ρομπότ- *Minitaur quadruped* (βλ. αρχείο *minitaur.urdf*)

14. Λαμβάνουμε υπόψιν τη φυσική του προβλήματος;

Ναι, ρυθμίζοντας στον κώδικα την τιμή της παραμέτρου ***USE_PHYSICS*** (*rigid-body dynamics, contact torque, gravity*).

15. Τι είναι ένας κύκλος (*cycle*);

Πρόκειται για ένα πλήρες πέρασμα των βημάτων 4-10. Ειδικότερα, σε κάθε κύκλο:

- κάνουμε *sample candidate solutions*
- επιλέγουμε τις 12 καλύτερες βάσει *UCB*
- τρέχουμε προσομοιώσεις
- ενημερώνουμε το *archive* και το *GP* μοντέλο μας.

16. Πώς υπολογίζουμε σε ποιο κελί θα τοποθετήσουμε μια συμπεριφορά του ρομπότ;

Βάσει των εξισώσεων:

$$i = \text{floor} \left((\text{speed} - s_{\min}) * \frac{B_s - 1}{s_{\max} - s_{\min}} \right) \quad (4.3)$$

$$j = \text{floor} \left((\text{effort} - e_{\min}) * \frac{B_e - 1}{e_{\max} - e_{\min}} \right) \quad (4.4)$$

Όπου κάθε δείκτης αντιστοιχεί προφανώς σε ποιο *grid* του *descriptor space* ανήκει μια συμπεριφορά του ρομπότ. Επίσης:

Speed και *effort*: η μετρούμενη γραμμική-forward ταχύτητα κατά το *simulation* και το *CoT* (**cost of transport**) αντιστοίχως.

$s_{\min}, s_{\max}, e_{\min}, e_{\max}$: τα αντίστοιχα άνω και κάτω όρια

$$B_s = 25 \text{ (αριθμός speed bins)} \quad (4.5)$$

$$B_e = 25 \text{ (αριθμός effort bins)} \quad (4.6)$$

17. Βάσει του .csv αρχείου, πώς το αξιοποιεί το ρομπότ προκειμένου να κινηθεί;

Τα *amplitudes* τα οποία είναι αποθηκευμένα στο αρχείο αξιοποιούνται ως εξής από τον παρακάτω ψευδοκώδικα:

```
for t in time_range:
```

```
     $\vartheta_i = A_i * \sin(2\pi * t /$   
SIM_DURATION)
```

```
    send  $\vartheta_i$  to motor i
```

$$\tau = K_p \frac{(\theta_{\text{target}} - \theta)}{dt} + K_d(\dot{\theta}_{\text{target}} - \dot{\theta})$$

(4.7)

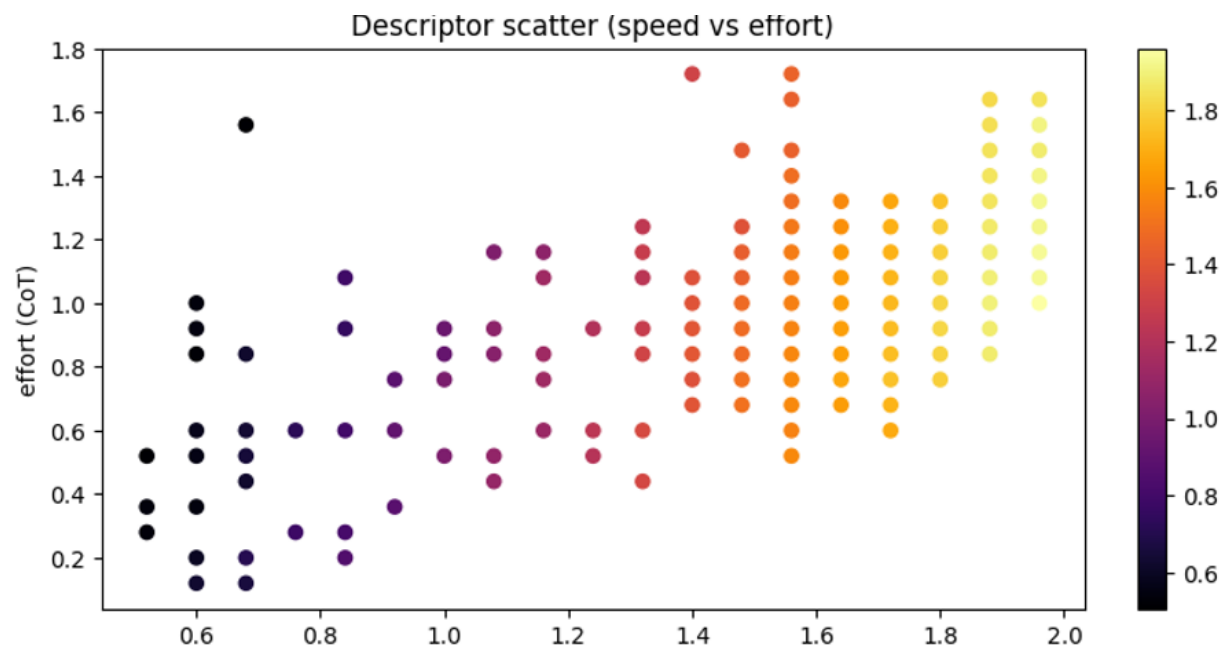
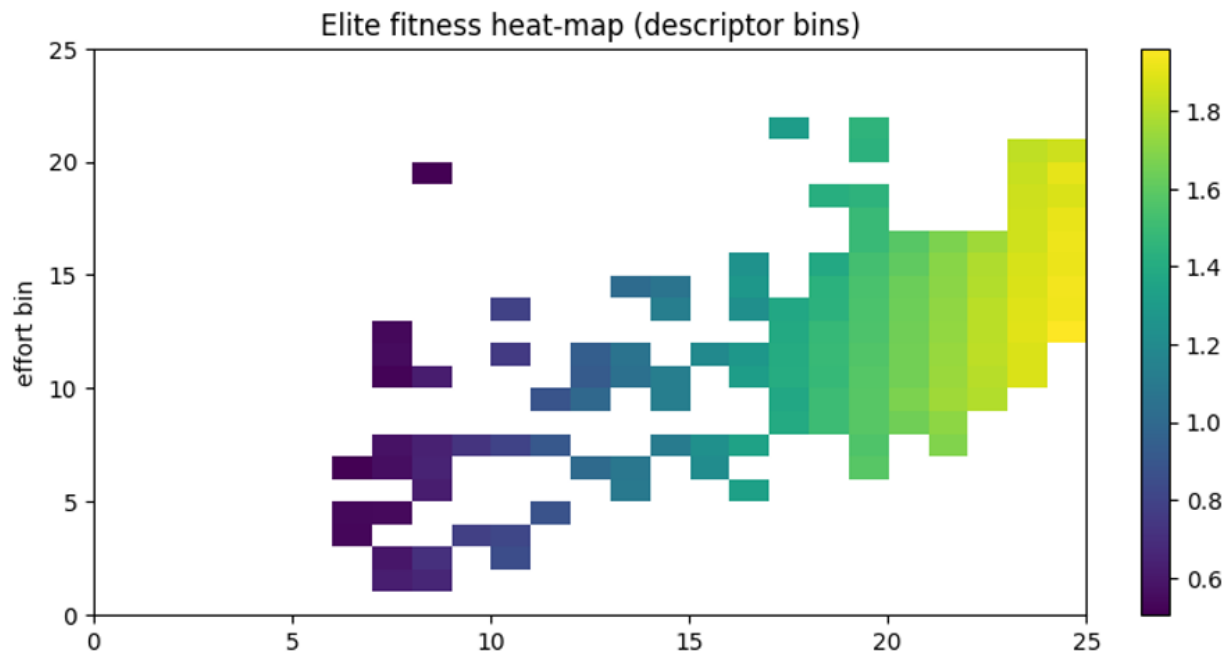
Μέσω της γραμμής:

```
p.setJointMotorControl2(robot_id, joint_i,  
controlMode=POSITION_CONTROL,  
targetPosition= $\theta_i$ )
```

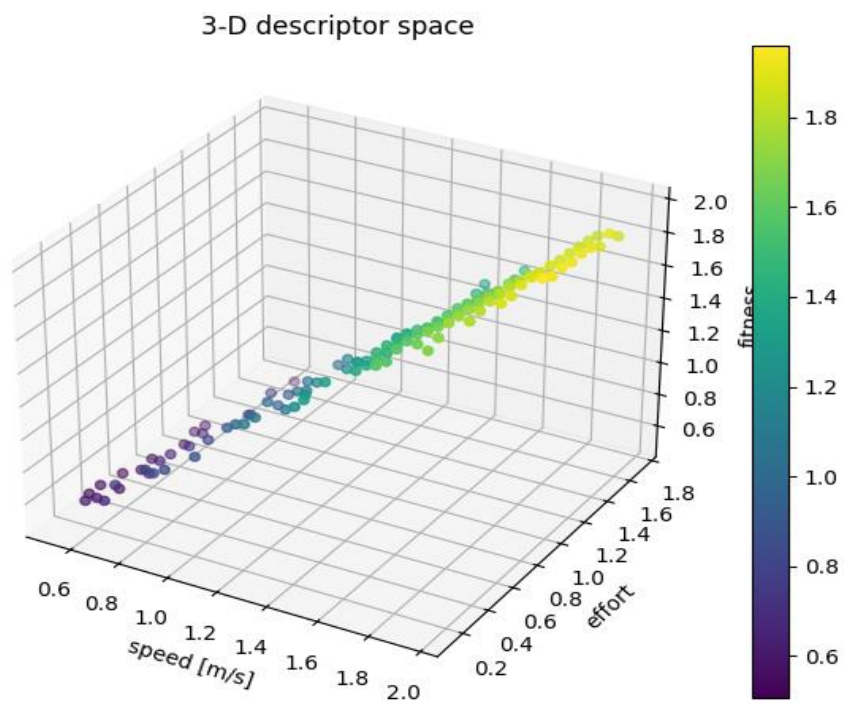
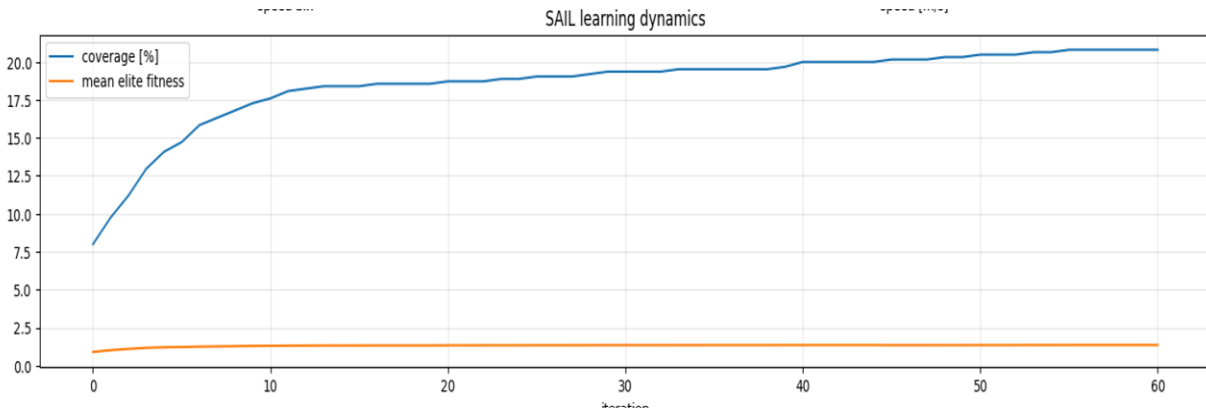
Με θ_{target} να είναι η γωνία που υπολογίζεται από το την σχέση: $\theta_{\text{target}} = A_i * \sin\left(2\pi * \frac{t}{T}\right)$ (4.8) και θ η «πραγματική» γωνία όπως εκείνη προκύπτει από την σχετική προσομοίωση. Οπότε, στέλνουμε τα *torque commands* όπως αυτά προκύπτουν από την (4.7) σε κάθε κινητήρα και έτσι κινείται το ρομπότ.

Στο .csv αρχείο έχουμε 128 σειρές, άρα και 128 διαφορετικές συμπεριφορές τις οποίες δύναται να ακολουθήσει το ρομπότ προκειμένου να «περπατήσει».

Τρέχοντας τον κώδικα (*max 1 λεπτό παίρνει*) λαμβάνουμε τα παρακάτω γραφήματα:



(οι οριζόντιοι άξονες αντιστοιχούν στην *forward velocity* του ρομπότ)



Σημείωση: Ο κώδικας δημιουργήθηκε εξ'ολοκλήρου από το Chat-GPT αλλά με πολύωρο prompting.

5-Discussion

Πλεονεκτήματα SAIL

- Εκρηκτική μείωση κόστους αξιολογήσεων**: Παράγει εκατοντάδες λύσεις με έως και 1000 λιγότερες πραγματικές προσομοιώσεις από τον καθαρό *MAP-Elite*.
- Χάρτης ποικιλίας + ποιότητας**: Γεμίζει ένα πλέγμα με υψηλής απόδοσης λύσεις ώστε ο χρήστης να διαλέξει ανά περίπτωση ανάγκης, όχι απλώς το 1 βέλτιστη λύση.
- Εκμετάλλευση αβεβαιότητας**: Ο κανόνας *UCB* καθοδηγεί πού «ξέρουμε λίγο» και άρα αξίζει να δοκιμάσουμε (*exploration and exploitation*).
- Ενσωματώνεται εύκολα** σε σύνθετες προσομοιώσεις επειδή ο *GP* απαιτεί μόνο ζεύγη εισόδου-εξόδου.
- Ευελιξία kernel**: Το ***Matérn 5/2*** μοντελοποιεί φυσικά φαινόμενα καλύτερα από τα υπερ-ομαλά *RBF*.

Μειονεκτήματα SAIL

- **Κλιμάκωση $O(N^3)$** : Ο χρόνος εκπαίδευσης *GP* γίνεται απαγορευτικός πάνω από ~1000 δείγματα.
- Ευαισθησία σε θόρυβο**: Αν το ***WhiteNoise*** στήθει λάθος, το *GP* γίνεται *overconfident* ή το ακριβώς ανάποδο (*overly scared*).
- «Κακό» αρχικό *variance* -κακή κατεύθυνση**: 60 τυχαία δείγματα πρέπει να καλύψουν κάπως όλο το εύρος, αλλιώς το *UCB* «στραβώνει».
- Περιορισμός σε λίγες διαστάσεις**: Πολλοί *descriptors* ή μεγάλο *genotype* κάνουν το πλέγμα εκθετικά μεγάλο.
- Υπόθεση ομαλότητας**: Πολύ «noisy» *fitness functions* δυσκολεύουν το *GP*.

Διάφορες βελτιώσεις που μπορούν να γίνουν

1. Scale the surrogate model

-Sparse/inducing-point Gaussian Processes

-Local or partitioned GPs

2. Εξυπνότερο exploration/exploitation

-Adaptive β schedule for UCB

-Adaptive batch size

3. Καλύτερο αρχικό sampling των δειγμάτων

-Latin-Hypercube/Sobol DoE

4. Πιο «έξυπνοι»/προσαρμοστικοί behavioural descriptors

πχ slip, stability, vibration

5. Πιο εξελιγμένοι ελεγκτές **Central Pattern Generators (CPGs)** ή **RL-modulated CPGs**

6. **Multiprocessing/multithreading/GPU processing** για την φυσική προσομοίωση.

Συμπεράσματα/Κλείσιμο

Ο *SAIL* μετατρέπει τη βελτιστοποίηση βηματισμού του ρομπότ μας σε ένα πρόβλημα σε χαρτογράφησης λύσεων. Έτσι, αντί να μας δίνει ένα μοναδικό «καλύτερο» βάδισμα, γεμίζει ένα πλέγμα όπου κάθε κελί κρατά την ταχύτερη, την πιο οικονομική ή μια ισορροπημένη κίνηση για εκείνη τη ζώνη. Εμείς στη εν συνεχεία διαλέγουμε ό,τι μας βολεύει. Στο σημείο αυτό να αναφερθεί ότι ένα από τα **key-components** του εν λόγω αλγορίθμου είναι το *Gaussian Process*· **με τον πυρήνα Matérn 5/2 να αποτυπώνει ομαλά αλλά και «κοφτά» σημεία της πραγματικής συνάρτησης, ενώ το UCB οδηγεί την αναζήτηση όπου είτε περιμένουμε υψηλό fitness είτε έχουμε μεγάλη άγνοια (*exploration vs exploitation*)**. Κατά συνέπεια, χρειαζόμαστε εκατοντάδες και όχι χιλιάδες «ακριβές» φυσικές προσομοιώσεις· στο δικό μας δε ρομπότ το οποίο και προσομοιώνουμε, 720 *simulations* δίνουν 128 ξεχωριστά μοτίβα βηματισμού του ρομπότ. Σε κάθε περίπτωση βέβαια, τα όποια μειονεκτήματα του *αλγορίθμου* όπως ο κυβικός χρόνος *GP* και το εκθετικής πολυπλοκότητας πλέγμα αντιμετωπίζονται (μέχρι ένα βαθμό) με *sparse GPs* και προσαρμοζόμενα πλέγματα *CVT* ή *quadtrees* ενώ ένα *Central Pattern Generator* αντί για απλά ημίτονα υπόσχεται φυσιολογικότερες και ίσως οικονομικότερες κινήσεις. Συνολικά, **ο *SAIL* δείχνει ότι μπορούμε να εξερευνήσουμε** ένα σχετικά μεγάλο μέρος του χώρου καταστάσεων **με λίγα σχετικά πειράματα, ανοίγοντας με αυτόν τον τρόπο το δρόμο σε ακόμη πιο μεγάλα και απαιτητικά προβλήματα Ρομποτικής**.

6-Παρατηρήσεις

Το αρχικό πλάνο ήταν η εργασία να γίνει από κοινού και από τα δύο μέλη της ομάδας, αλλά λόγω περιορισμένου χρόνου το ένα μέλος αποφάσισε να μη ασχοληθεί πλέον άλλο με την εργασία (έγιναν προσπάθειες εκ μέρους του να προσομοιωθεί ένα πιο ρεαλιστικό ρομπότ σε PyBullet αλλά δε βγήκε κάποιο ουσιαστικό αποτέλεσμα και έτσι ενώ είχε συμφωνηθεί εκ των προτέρων ότι θα εργαζόταν σε αυτό το κομμάτι, λίγες μέρες πριν το τελικό deadline (και αφότου είχαμε συναντηθεί/συζητήσει/μελετήσει σχετικά resources για το τι θα κάνουμε στην εργασία), μου έκανε γνωστό ότι μάλλον δεν θα ασχοληθεί άλλο ενώ τη δεδομένη στιγμή είχαμε έτοιμο μόνο ένα σχετικά απλό script (δεν είχαμε αρχίσει καν κάποια αναφορά/πόσω δε μάλλον κάποια παρουσίαση) το οποίο και εφαρμόζε τον αλγόριθμο με τον οποίο καταπιάνεται η παρούσα εργασία. Έτσι, αναγκάστηκα μέσα σε λίγες μέρες (3) να συνεχίσω με την εργασία αποκλειστικά μόνος μου (είχε ειδοποιηθεί ότι πλησιάζει η προθεσμία και πράγματι το παιδί προσπαθούσε να προσομοιώσει ένα ρομπότ στην PyBullet, αλλά). Επομένως, η εργασία στην ολότητά της (κώδικας, word, powerpoint) εκπονήθηκε εξ'ολοκλήρου από το ένα μέλος της ομάδας. Έγινε προσπάθεια να συμπεριληφθεί και το live animation του ρομπότ βάσει των διάφορων generated συμπεριφορών αλλά αυτό δεν κατέστη εφικτό για ευνόητους λόγους. Όπως επίσης και έγινε προσπάθεια σύγκρισης του implementation του αλγορίθμου που υπάρχει στον κώδικα με την υλοποίηση που υπάρχει και στη βιβλιοθήκη qdrg, ωστόσο ούτε και αυτή η επέκταση κατέστη δυνατή. Δηλαδή, υπήρχαν διάφορες ιδέες και από τα δύο μέλη της ομάδας αλλά οι περισσότερες από αυτές δεν πραγματοποιήθηκαν για τους προαναφερθέντες λόγους.

7-Ενδεικτική Βιβλιογραφία που χρησιμοποιήθηκε στην εργασία

1. Gaier, A., Asteroth, A., & Mouret, J.-B. (2018). *Data-Efficient Design Exploration through Surrogate-Assisted Illumination*. arXiv:1806.05865
2. Mouret, J.-B., & Clune, J. (2015). *MAP-Elites: Illuminating Search Spaces by Mapping Elites*. arXiv:1504.04909
3. Vassiliades, V., Chatzilygeroudis, K., & Mouret, J.-B. (2016). *Using Centroidal Voronoi Tessellations to Scale Up MAP-Elites*. arXiv:1610.05729
4. Gallotta, R., Liapis, A., & Yannakakis, G. N. (2024). *Dynamic Quality-Diversity Search*. arXiv:2404.05769
5. Srinivas, N., Krause, A., Kakade, S., & Seeger, M. (2010). *Gaussian Process Optimization in the Bandit Setting: No-Regret and Experimental Design*. ICML Proceedings
6. Titsias, M. K. (2009). *Variational Learning of Inducing Variables in Sparse Gaussian Processes*. JMLR Vol. 5
7. Bonilla, E., Chai, K. M. A., & Williams, C. K. I. (2008). *Multi-Task Gaussian Process Learning of Robot Inverse Dynamics*. NeurIPS
8. Ye, Y. et al. (2022). *Learning Central Pattern Generators for Quadruped Locomotion*. arXiv:2211.00458
9. IEEE Spectrum Staff. (2016). *Ghost Robotics' Minitaur Quadruped Hits 2 m/s*. IEEE Spectrum Online Article
10. Bertsimas, D. et al. (2023). *Sparse Inducing Points in Deep Gaussian Processes*. arXiv:2407.17033
11. Lundell, K., & Nikolaidis, S. (2025). *A Quality-Diversity Approach to Evolving Model Rockets*. arXiv:2504.02177
12. Rakotoarison, H. et al. (2019). *Automated Discovery of Behaviour Descriptors with VQ-VAE in QD Algorithms*. (Conference paper)
13. [Tutorial on Quality-Diversity Optimisation](#)
14. [ALIFE2023: Efficient Exploration using Model-Based Quality-Diversity with Gradients](#)
15. [List of papers | Quality-Diversity optimisation algorithms](#)